

Supply Chain Security Report

End-to-end artifact trust, image signing, SBOM, provenance, attestation, exception governance, and admission controls.

Field	Value
Project	Enterprise DevSecOps Security Lab
Document Owner	Jagriti Banerjee
Document Type	Supply Chain Security Report
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions, GHCR
Classification	Portfolio / Private Lab Evidence
Status	Final Detailed Spacing Fixed

Document Scope

This report is part of the Enterprise DevSecOps evidence package. It is written for technical reviewers and recruiters who want to understand how artifact trust, SBOM visibility, provenance, VEX notes, and exception governance were implemented and validated in the private lab.

Reviewer Focus Area	What this document proves
Artifact trust	Shows how image signing, digest review, and GHCR publishing protect artifact integrity.
Metadata transparency	Documents SBOM and provenance generation, attachment, attestation, and verification.
Governance	Defines security gates, exception handling, VEX notes, and maturity roadmap.

Evidence Package	Included
Supply chain controls	Cosign signing, Syft SBOM, provenance attestation, Gatekeeper metadata controls
Decision records	Security gate policy, exception rules, VEX status notes, retest expectations
Release quality focus	Artifact integrity, component transparency, vulnerability triage, and audit readiness

Executive Summary

The Enterprise DevSecOps project implements a multi-layer supply chain security workflow for a containerized Flask application. The workflow covers source-to-image traceability, GHCR image publishing, Cosign image signing, Syft SBOM generation, SBOM and provenance attestations, Kubernetes admission metadata checks, and formal exception/VEX documentation. The objective is to prove that the project can move from vulnerable application testing to controlled release evidence with traceable artifacts.

Assessment Conclusion

The project demonstrates portfolio-grade supply chain security maturity. The remaining production gap is to move from local lab signing and local provenance generation to CI-generated, identity-bound, digest-pinned, admission-verified release controls.

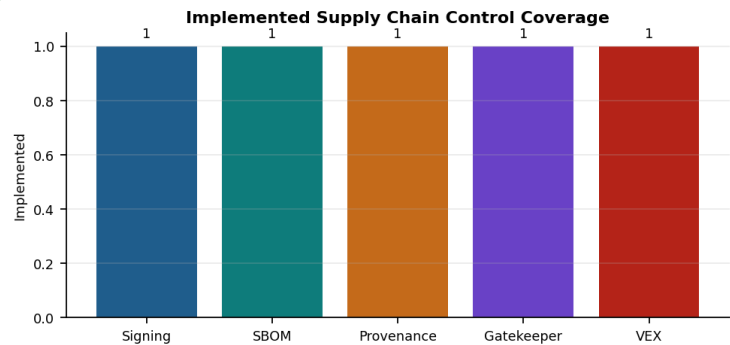


Figure 1 - Supply chain control areas implemented in the lab.

Supply Chain Architecture and Trust Flow

The lab follows a defined chain of trust. Source code is committed to Git, built into a Docker image, pushed to GHCR, signed with Cosign, inventoried with Syft, linked to provenance metadata, and deployed into Kubernetes with admission governance. Each step produces evidence that can be reviewed independently.

Control Area	Implemented Control	Evidence	Security Value
Image signing	Cosign key-pair signature for GHCR image	cosign-image-verify.txt; Cosign verification screenshot	Establishes artifact authenticity and tamper evidence.
SBOM generation	Syft SPDX and CycloneDX SBOMs	sbom-spdx.json; sbom-cyclonedx.json	Creates package inventory for vulnerability response and audits.
SBOM attestation	Cosign signed SBOM attestation	sbom-attestation-verify.txt	Links the SBOM to the image with signed metadata.
Provenance	SLSA-style provenance predicate	slsa-provenance.json; provenance verification report	Documents source, commit, builder, digest, and build timeline.
Admission metadata	Gatekeeper required labels: security-owner, sbom, signed	Gatekeeper blocked/allowed evidence	Prevents unlabeled workloads from entering the namespace.
Exception governance	Vulnerability exception and VEX note process	Vulnerability Exceptions and VEX Notes documents	Separates real remediation from temporary risk acceptance.

Threat Model for the Artifact Path

Threat Scenario	Potential Impact	Control Implemented	Residual Risk
Attacker tampers with image after build	Compromised workload enters Kubernetes.	Cosign signature verification and digest inspection.	Admission-time signature verification not yet enforced.

Threat Scenario	Potential Impact	Control Implemented	Residual Risk
Dependency with critical CVE is added	Known exploitable component ships in image.	pip-audit, Trivy, SBOM, security gate policy.	Intentional vulnerable dependency remains for lab evidence.
SBOM is stale or mismatched	Incident response uses wrong inventory.	SBOM regenerated from image and attested.	Must be automated in CI for production.
Build provenance is forged	Reviewer cannot trust origin metadata.	Cosign signed SLSA-style provenance.	Local builder identity is not as strong as CI/OIDC identity.
Unlabeled deployment bypasses governance	Release lacks owner/SBOM/signing context.	OPA Gatekeeper required-label policy.	Policy checks metadata, not cryptographic signature.

Security Gate Policy

The project uses explicit blocking rules for findings that create immediate or high-confidence supply chain risk. These gates are meant to stop release promotion, not merely create warnings.

Gate Condition	Decision	Reason
Critical dependency vulnerability	Fail	Dependency issues can be exploitable before runtime hardening helps.
Critical container vulnerability	Fail	Base image or OS packages may compromise the image execution boundary.
Secret found	Fail	Secrets in source or image create immediate credential exposure risk.
Dockerfile critical misconfiguration	Fail	Build and runtime misconfiguration can weaken isolation and provenance.
Kubernetes manifest critical misconfiguration	Fail	Critical workload misconfigurations can create cluster compromise paths.

Validation Evidence Register

Evidence Artifact	How It Is Used	Reviewer Check
cosign-image-verify.txt	Confirms image signature verification.	Public key verification succeeds for expected image.
sbom-spdx.json / sbom-cyclonedx.json	Provides component inventory.	File is generated after the image build and matches current digest.
sbom-attestation-verify.txt	Confirms signed SBOM attestation.	Cosign attestation verification succeeds.
slsa-provenance.json	Documents builder, source, ref, and digest.	Predicate includes repository, ref, digest, and builder identity.
slsa-provenance-verify.txt	Confirms provenance attestation verification.	Verification succeeds using expected public key.
Gatekeeper block/allow screenshots	Proves admission metadata policy.	Non-compliant deployment is blocked; compliant deployment is allowed.

Vulnerability and Exception Governance

The lab separates vulnerability discovery from release approval. A scanner result may identify a package, but release decisions require triage, VEX context, exception rules, owner assignment, and retest criteria. This prevents exceptions from becoming invisible permanent risk.

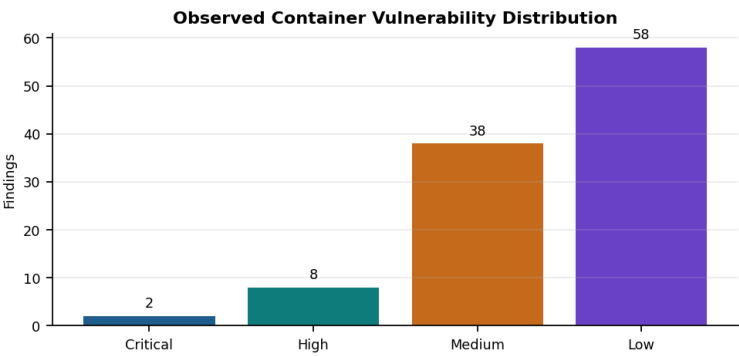


Figure 2 - Example vulnerability severity distribution used for gate decisions.

Decision	Allowed in Private Lab	Allowed for Production Release	Required Evidence
Intentional vulnerable Flask route	Yes	No	PoC screenshot and lab-only disclaimer.
Critical dependency vulnerability	Yes, for evidence only	No, unless exception approved temporarily	pip-audit/Trivy output, owner, expiry, fix plan.
Signed image with known vulnerable package	Yes	Only after triage	Signature, SBOM, VEX note, remediation plan.
Missing SBOM or provenance	No	No	Regenerate SBOM/provenance and verify attestation.

Maturity Roadmap

Maturity Step	Current State	Next Improvement
Level 1 - Visibility	SBOMs, scans, signatures, and reports exist.	Automate evidence creation in CI.
Level 2 - Integrity	Cosign signatures and attestations are verified manually.	Deploy only signed image digests.
Level 3 - Policy Enforcement	Gatekeeper enforces labels.	Enforce image signatures using Sigstore Policy Controller or Kyverno.
Level 4 - Continuous Governance	Exception and VEX process documented.	Automate exception expiry and dashboard visibility.

References

- CISA VEX minimum requirements: <https://www.cisa.gov/sites/default/files/2023-04/minimum-requirements-for-vex-508c.pdf>
- OpenVEX specification: <https://github.com/openvex/spec/blob/main/OPENVEX-SPEC.md>
- SLSA specification: <https://slsa.dev/spec/v1.2/>
- SLSA provenance specification: <https://slsa.dev/spec/v1.0/provenance>
- Sigstore Cosign project: <https://github.com/sigstore/cosign>
- Cosign specifications and attestation support: https://docs.sigstore.dev/cosign/system_config/specifications/
- Sigstore project overview: <https://www.sigstore.dev/>